

PARTE 1

1/2 somador

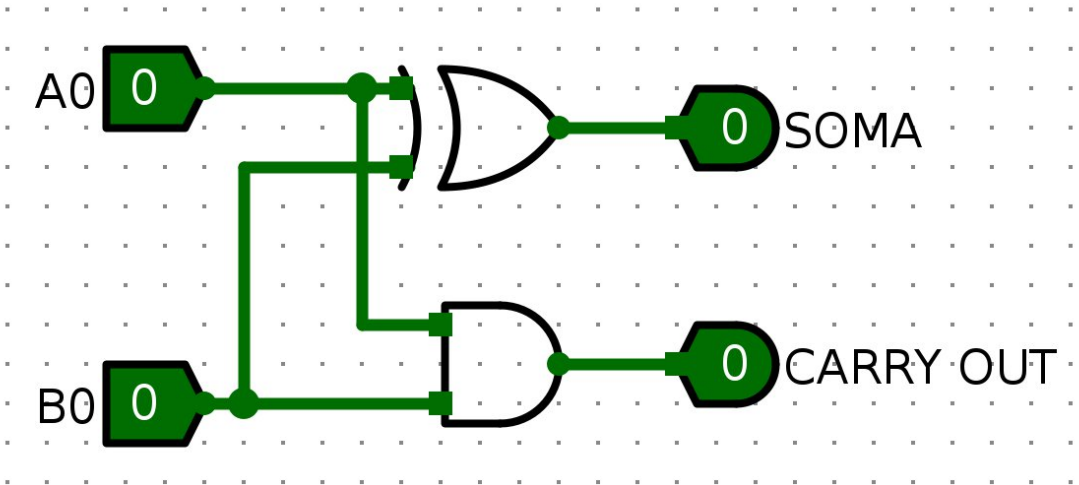


Tabela-verdade

A	B	S	COUT
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Somador completo de 1 bit

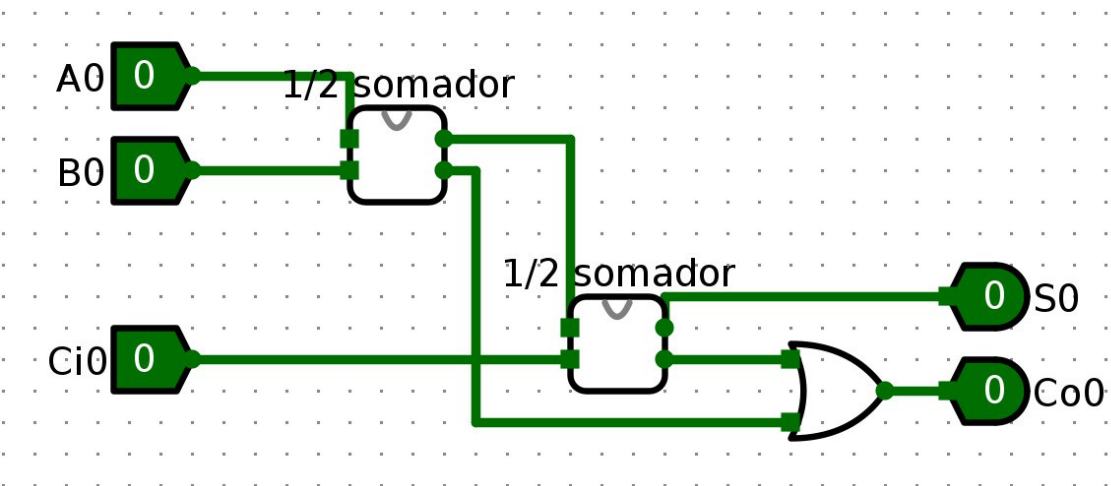
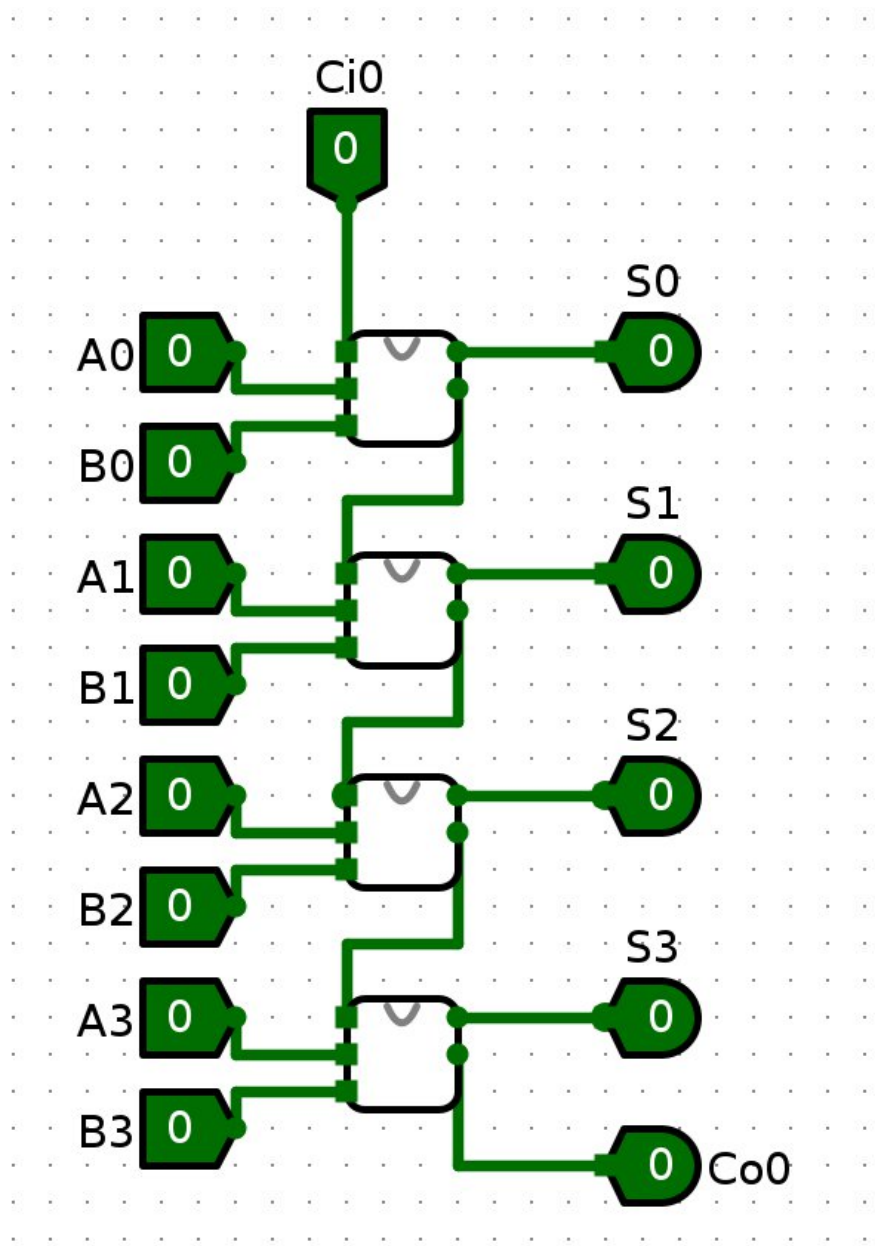


Tabela-verdade

A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Somador completo



Perguntas:

1) Qual o problema de tempo associado a esse tipo de somador (pense no carry), considere o atraso médio de cada porta lógica de 10 ns.

R: O problema é o atraso de propagação do carry produzido pelo bit anterior. O bit mais significativo precisa esperar o carry atravessar os estágios anteriores. Dessa forma, quanto maior o número de bits do somador, maior será o tempo necessário para obter o resultado final.

2) Qual o tempo necessário para a computação de uma soma e do vai um em um somador de 4 bits.

R: $Co0=30$, $Co1=60$, $Co2=90$, $Co3=120$ ns. Soma final (S3): $90+10 = 100$ ns. Carry-out final: 120ns.

3) O que seria necessário para um somador de 32 bits?

R: Encadear 32 somadores completos de 1 bit. O carry-out de cada estágio seria conectado ao carry-in do estágio seguinte.

4) Considerando esses tempos acima, calcule a frequência de operação de um somador de 32 bits.

R: Cada estágio: 30ns $\rightarrow 32 \times 30 = 960$ ns (carry final). Soma final (S3): $90+10 = 100$ ns. Carry-out final: 120ns.

$$f=1/960\text{ns}$$

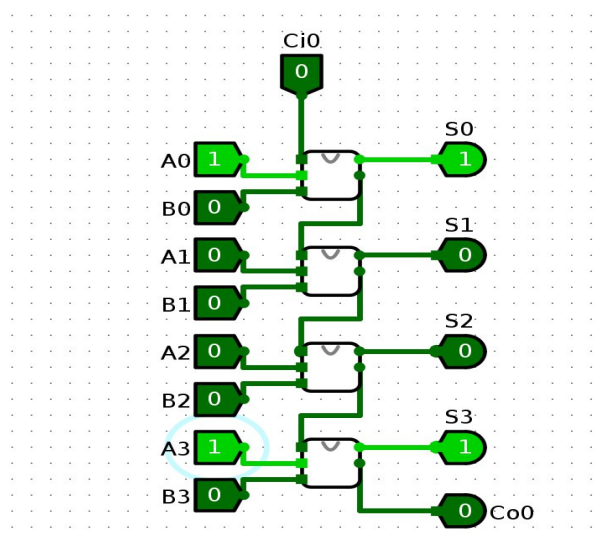
$$f=1,04\text{MHz}$$

5) Você consegue propor alguma forma de tornar essa soma mais veloz?

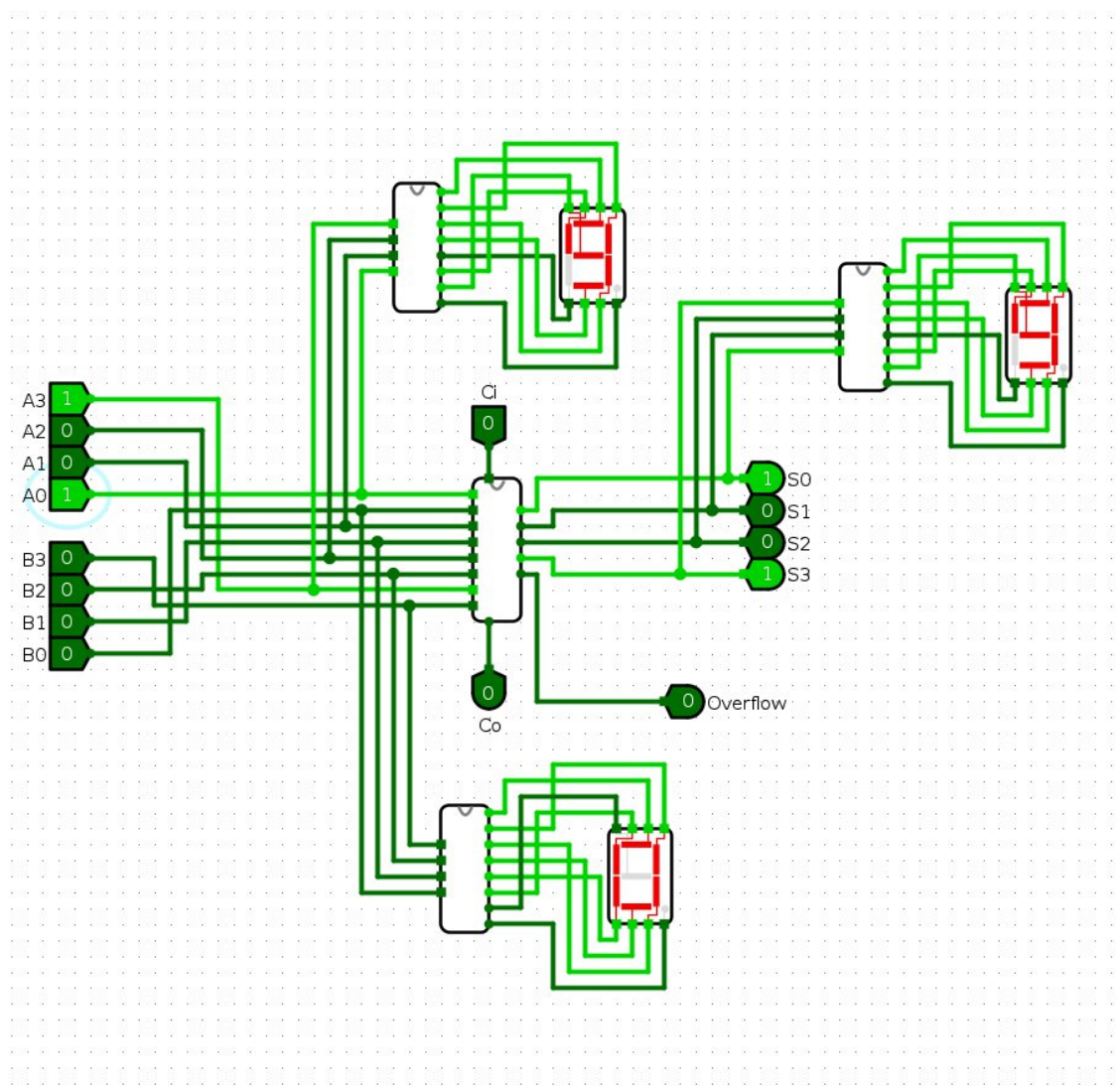
R: Uma forma de tornar a soma mais veloz seria utilizar um somador com antecipação de carry. Ele calcula os carries antecipadamente, reduzindo o atraso causado pela propagação sequencial do carry entre os bits.

Mostre uma soma de 2 números de 4 bits, os dois primeiros dígitos do seu número de matrícula.

R: Os dois primeiros dígitos da matrícula 902665 são 9 e 0.



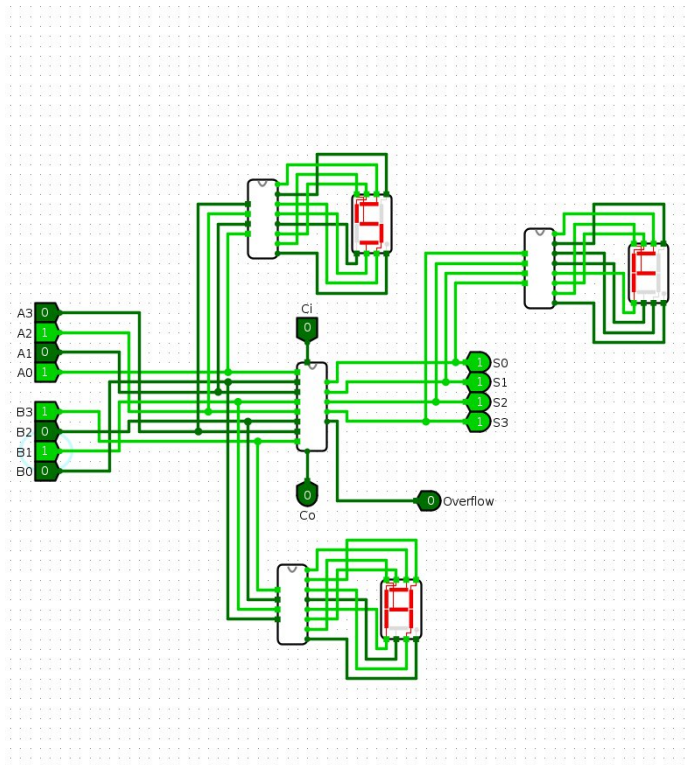
PARTE 2



Os dois primeiros dígitos da matrícula 902665 são 9 e 0.

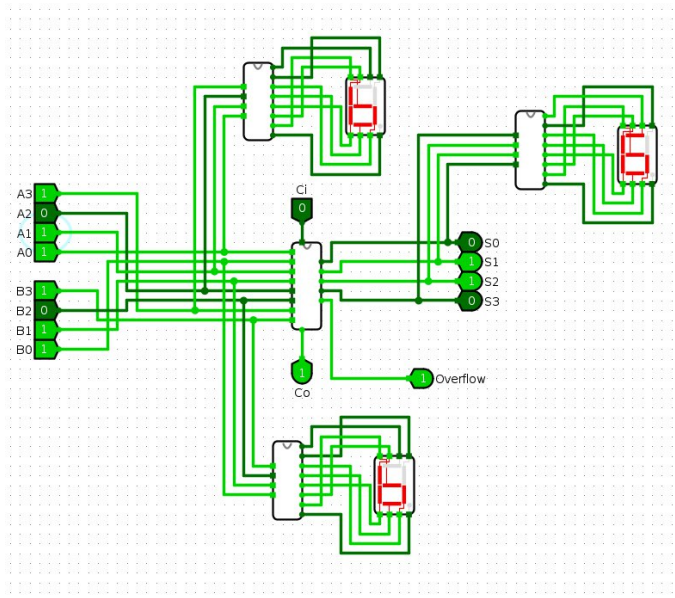
PARTE 3

A) $+5 + (-6)$



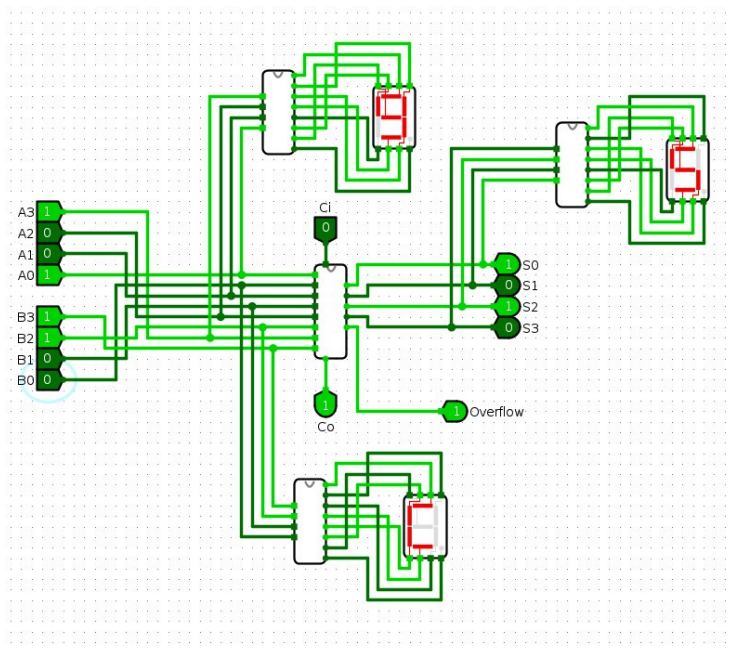
DISPLAY: 5 , A , F

B) $(-5) + (-5)$



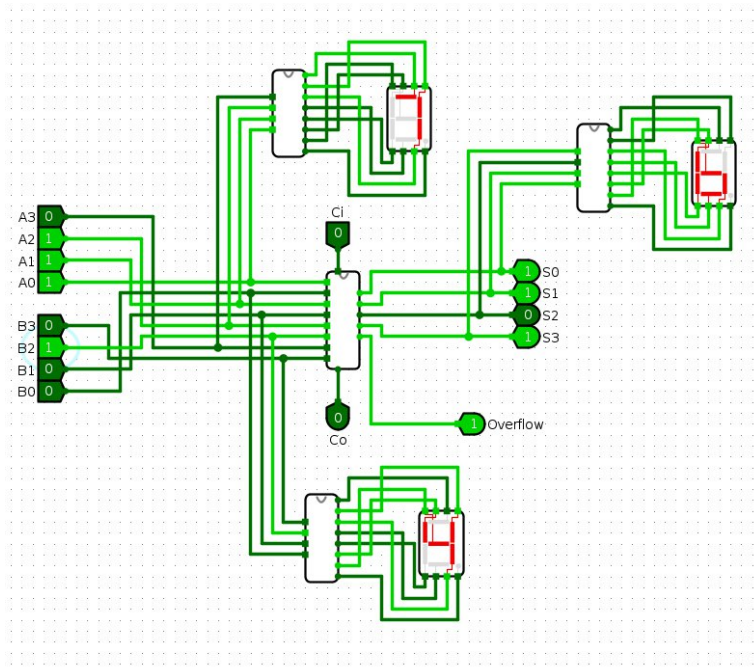
DISPLAY: B , B , 6

C) $(-7) + (-4)$



DISPLAY: 9 , C , 5

D) $+7 + 4$



DISPLAY: 7 , 4 , B

PERGUNTAS

1. Porque as duas últimas operações irão ligar o overflow?

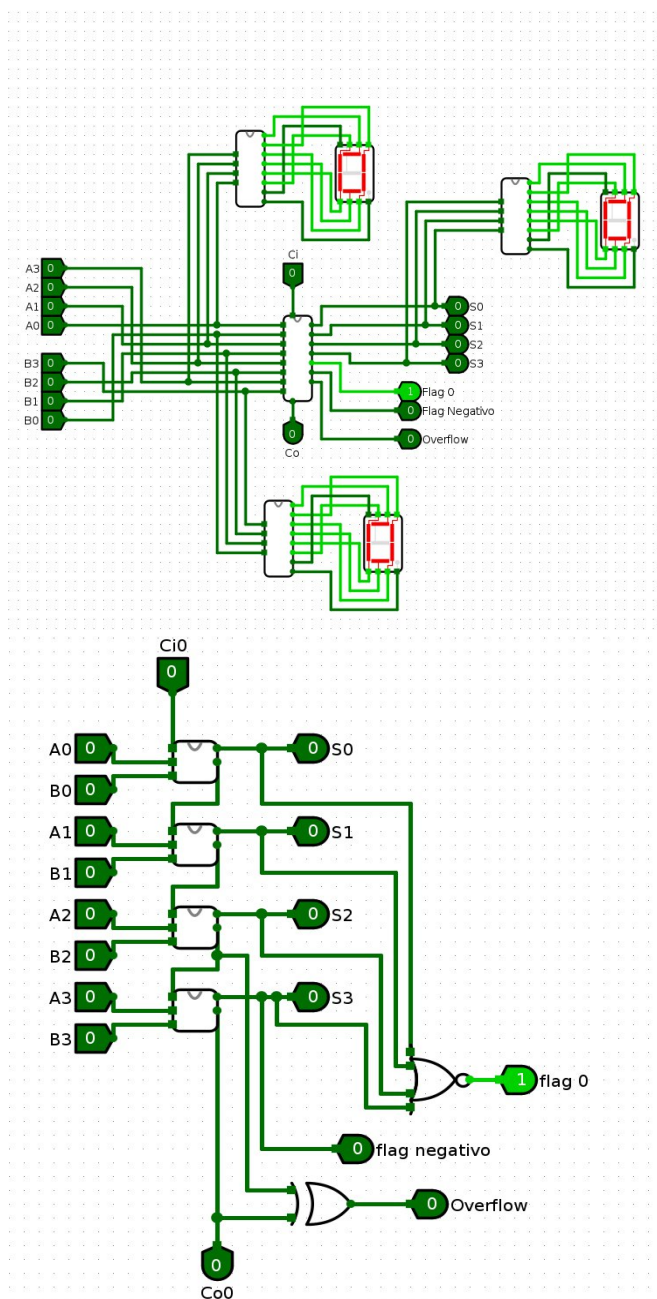
R: As operações c) e d) geram overflow porque o resultado matemático não pode ser representado utilizando apenas 4 bits em complemento de dois. Na operação c), $-7 + (-4) = -11$, que é menor que o limite mínimo de -8. Na operação d), $+7 + 4 = +11$, que é maior que o limite máximo de +7.

2. Qual a porta lógica que você utilizou para esta operação de identificar overflow?

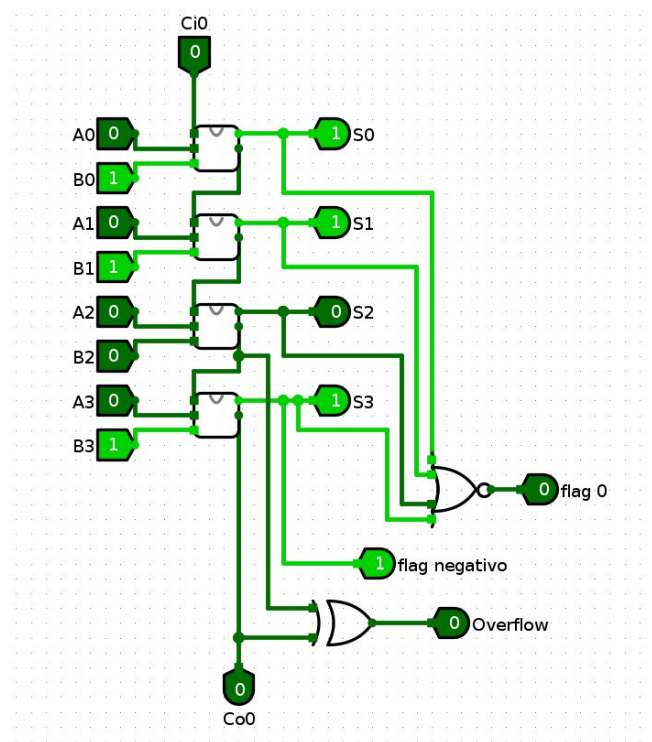
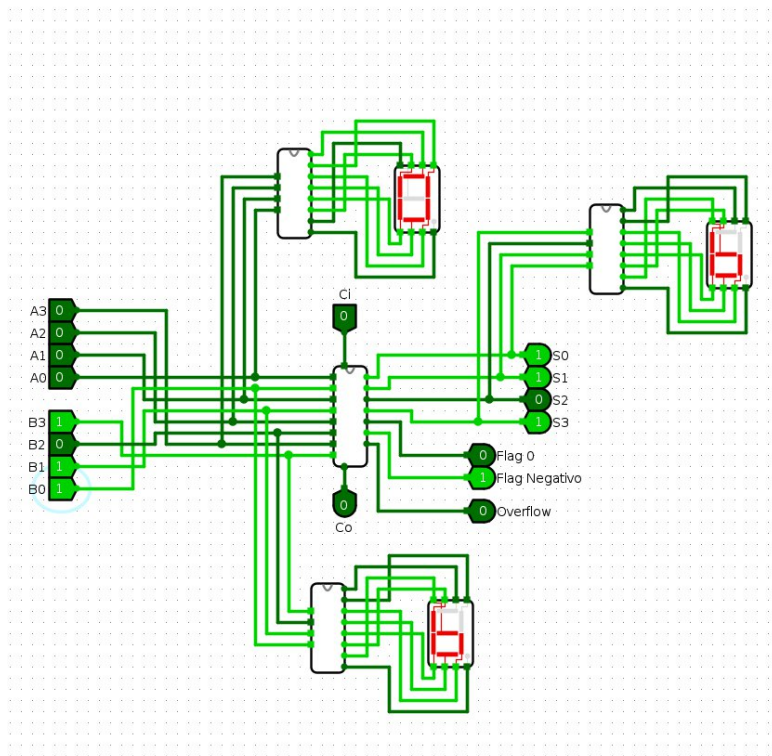
R: Foi utilizada uma porta lógica XOR, comparando o carry de entrada e o carry de saída do bit mais significativo.

PARTE 4

FLAG ZERO



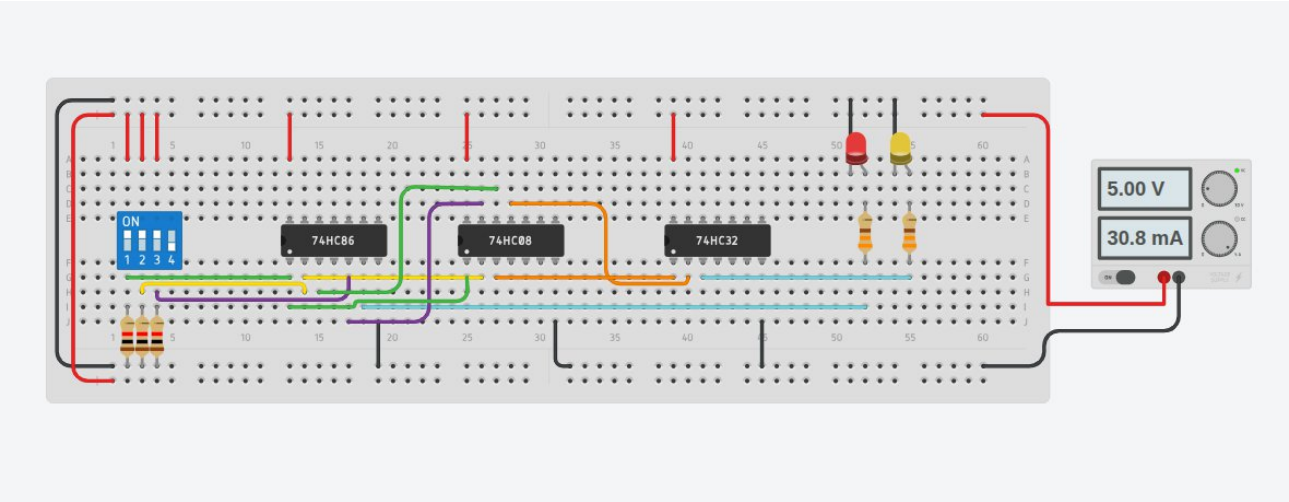
FLAG NEGATIVO



Você consegue pensar em alguma situação onde esses flags possam ser utilizados?

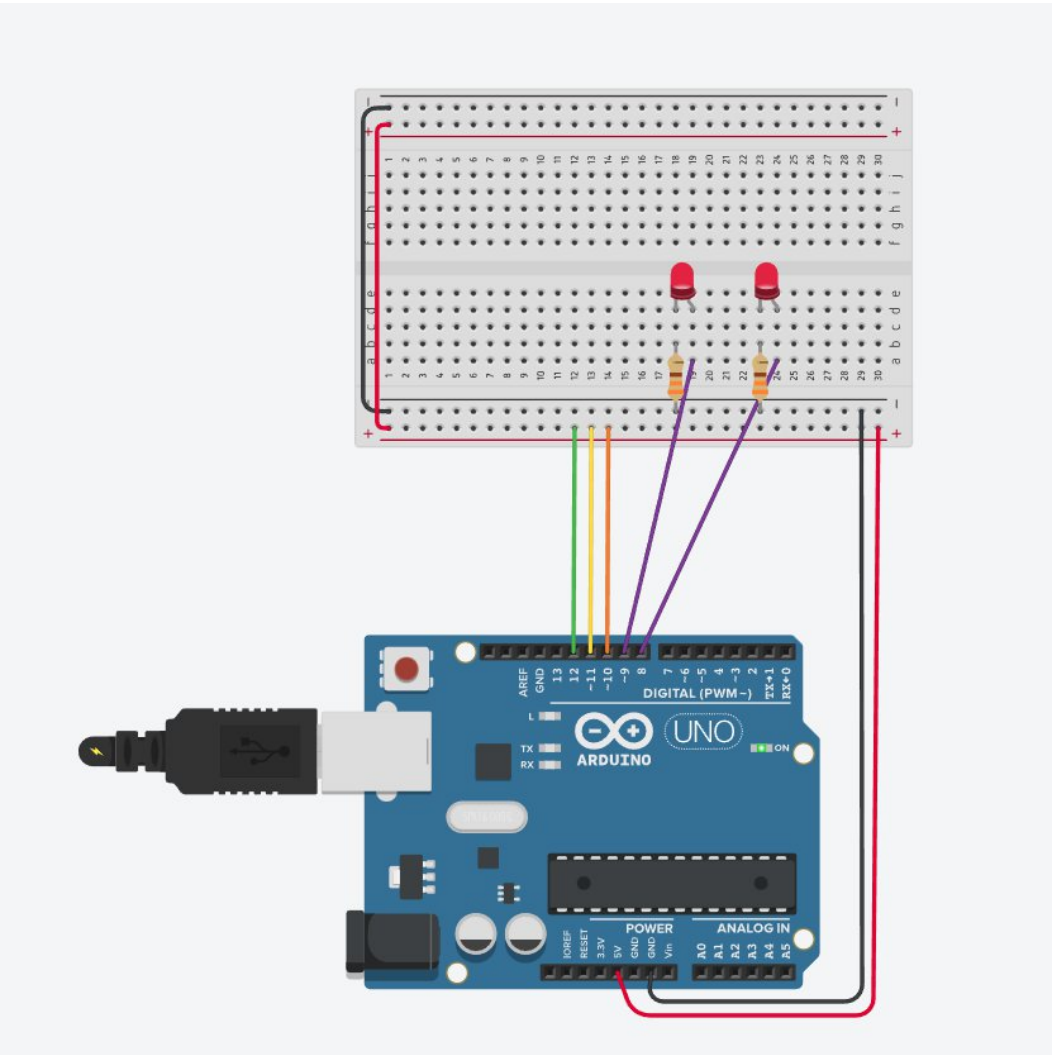
R: Esses flags podem ser utilizados em processadores para verificar rapidamente o resultado de uma operação. O Flag Zero pode indicar que o resultado de uma comparação ou operação foi zero, enquanto o Flag Negativo pode indicar que o resultado de uma operação aritmética é negativo.

PARTE 5



PARTE 6

ARDUÍNO:



CÓDIGO:

```
1 // C++ code
2 // Samuel Fragoso - 902665
3 int pino_a = 12;
4 int pino_b = 11;
5 int pino_cin = 10;
6 int pino_soma = 8;
7 int pino_cout = 9;
8
9 int valor_a, valor_b, valor_cin;
10 int valor_soma, valor_cout;
11
12 int portaxor(int x, int y) {
13     return (x != y) ? 1 : 0;
14 }
15
16 int portaand(int x, int y) {
17     return (x & y);
18 }
19
20 int portaor(int x, int y) {
21     return (x | y);
22 }
23
24 void setup() {
25     pinMode(pino_a, INPUT);
26     pinMode(pino_b, INPUT);
27     pinMode(pino_cin, INPUT);
28     pinMode(pino_soma, OUTPUT);
29     pinMode(pino_cout, OUTPUT);
30 }
31
32 void loop() {
33     valor_a = digitalRead(pino_a);
34     valor_b = digitalRead(pino_b);
35     valor_cin = digitalRead(pino_cin);
36
37     int x1 = portaxor(valor_a, valor_b);
38     valor_soma = portaxor(x1, valor_cin);
39
40     int y1 = portaand(valor_a, valor_b);
41     int y2 = portaand(x1, valor_cin);
42     valor_cout = portaor(y1, y2);
43
44     digitalWrite(pino_soma, valor_soma);
45     digitalWrite(pino_cout, valor_cout);
46 }
```